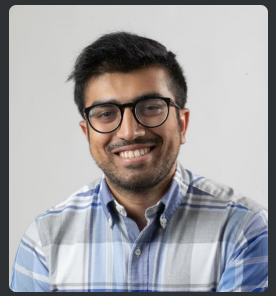
HPML

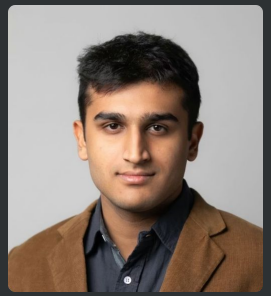
AgentOpsBench: High-Throughput Agentic AI for Battery Analytics

Extending IBM's AssetOpsBench to lithium-ion battery systems with MCP-based multi-agent orchestration, achieving 1.59× end-to-end speedup on fleet-scale RUL prediction through disk caching and tool batching.



Siddharth Gowda

Data Science Institute



Rushin Bhatt

Data Science Institute

Aryaman Agrawal

Dept. of Electrical Engineering

Winston Li

Data Science Institute

HPML Final Project · Spring 2026

GitHub: https://github.com/siddharthgowda/AssetOpsBench

Orginal Repo: https://github.com/IBM/AssetOpsBench

Dhaval Patel · IBM Mentor · Dr. Kaoutar El Maghraoui · Columbia University · Week of May 4, 2026

Motivation & Problem

Time budget: 1 minute.

Why this matters

- Lithium-ion batteries are critical in EVs, grid storage, aerospace, and UPS systems; battery degradation manifests through capacity fade, internal resistance growth, lithium plating, and thermal runaway
- AssetOpsBench is currently limited to HVAC assets and lacks battery telemetry, diagnostics, and prognostic tools
- Battery degradation is nonlinear and coupled, requiring domain-specific analytical tools and multi-step agentic reasoning

Problem statement

Extend IBM's AssetOpsBench to lithium-ion battery systems with MCP-based multi-agent orchestration, integrating NASA PCOE Li-ion cycling data (34 cells), pretrained DNN prognostics, and a 15-scenario benchmark.

Goal: Achieve practical performance on fleet-scale RUL prediction while maintaining composable, modular tool architecture with minimal modifications to upstream orchestration.

Technical Approach — Model & Data

Time budget: ~1 min of the 3-min approach section.

Model

Pretrained DNN (Hsu et al., Applied Energy 2022). Trained on 1 early cycle of charge/discharge data. Predicts RUL (cycles to 30% capacity fade) and voltage curves. Integrated via tf_keras compatibility layer for TF 2.16+. Runs on CPU with low latency inference per cell.

Dataset

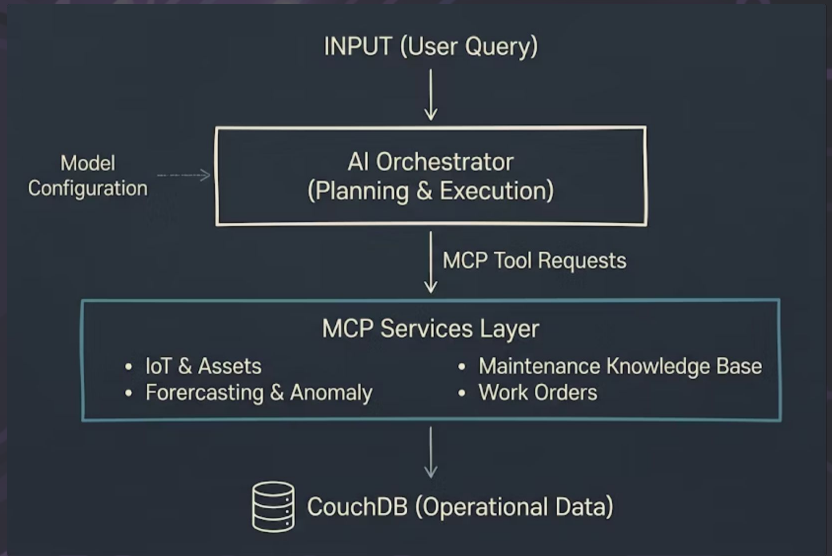
NASA PCOE Li-ion Battery Aging Dataset. 34 cells (18650 NCA chemistry). 10 cells with complete paired profiles (100+ cycles each). CouchDB telemetry layer with cycle-type partitioning. Charge, discharge, impedance measurement types.

Hardware

Apple M4 Max (ARM, 14 cores, 36 GiB unified memory). CPU-only, no GPU. Python 3.12.10, TensorFlow 2.21.0, FastMCP, CouchDB (Docker). LLM backend: Llama 3.1 (8B) via Cerebras.

Brief Overview of AssetOpsBench

AssetOpsBench is an open-source multi-agent benchmark for industrial asset operations. An LLM planner turns natural-language maintenance requests into multi-step plans, and an executor carries them out by calling domain-specific services. Our work extends this framework to battery systems.



Battery MCP Server – 11 Tools Total

- What We Built
  - Standalone MCP server added alongside existing AssetOpsBench servers (IoT, TSFM, FMSR, Work Orders)
  - 11 composable tool calls covering fleet management, prognostics, diagnostics, and statistical analysis
  - Discoverable by the planner like any other server. Mo special routing needed
- Tool Categories
  - Fleet listing and cycle summary retrieval
  - RUL prediction (single cell and fleet-batched)
  - Voltage curve and milestone prediction
  - Impedance trajectory and growth analysis
  - Capacity outlier detection and LLM-narrated diagnosis
- Prognostic Model
  - Lightweight pretrained DNN (Hsu et al.) Runs on CPU
  - Takes 100 early charge-discharge cycles as input
  - Predicts RUL and voltage behavior
  - Data: NASA PCOE Li-ion Battery Aging Dataset (10 cells, NCA 18650 chemistry)

Technical Approach — Optimizations Applied

Time budget: ~2 min. Name each optimization, give a 1-line intuition for why it should help.

01

Parallel CouchDB Prefetch

Overlap network requests via a thread pool to reduce data retrieval latency by 32% (5.7s → 3.9s for 10 cells).

02

Disk-Backed .npz Cache

Persist RUL trajectories (~7 KB per cell) to disk across MCP stdio cold-starts, eliminating redundant inference and reducing boot precompute from 7.0s to 0.002s.

03

MCP-Level Tool Batching

Consolidate fleet queries into single tool calls, reducing subprocess spawns from N to 1 and achieving 6× speedup for 10-cell queries.

Profiling Methodology

Tools used, what you measured, how you ensured measurements are stable.

Tools

- Wall-clock timing (CPU-bound system, no GPU profiler needed)
- MCP call latency instrumentation (per-tool, per-cell)
- CouchDB query timing via thread pool logs

Metrics captured

- Latency (wall-clock, p50/p95) and throughput (cells/sec)
- Per-stage breakdown: fetch (CouchDB) vs. predict (TensorFlow)
- Subprocess spawn overhead and cold-start cost
- Tool call count and batching efficiency

Results — Headline Numbers

Big numbers up front. Replace with your measured improvements.

1.59×

end-to-end speedup

Fleet Operator scenario (10-cell RUL ranking): 133.6s → 83.8s with disk cache

2.45×

tool-time reduction

Total tool execution: 78.1s → 31.9s with disk cache

6.0×

MCP batching speedup

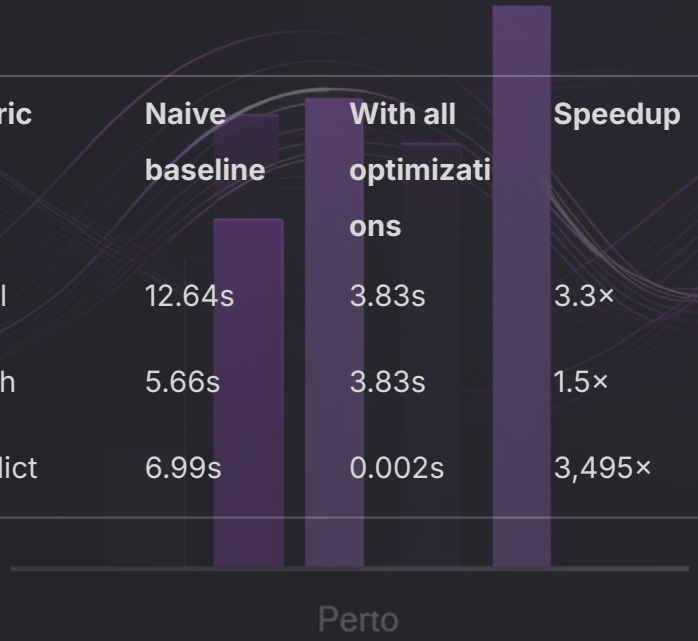
10-cell RUL prediction: 44.6s (per-cell) → 7.4s (batched)

All measurements on Apple M4 Max CPU, 3 repeats per configuration, wall-clock timing.

Results — Before vs. After

Actual measurements from the LaTeX report are shown below.

Metric	Naive baseline	With all optimizations	Speedup
Total	12.64s	3.83s	3.3×
Fetch	5.66s	3.83s	1.5×
Predict	6.99s	0.002s	3,495×



Takeaway

- Disk cache is the dominant optimization, reducing inference from 7s to 0.002s by eliminating redundant model loading across MCP subprocess boundaries.
- Parallel CouchDB fetch provides steady 32% improvement; graph precompilation and batched predict show negligible gains on this small model.
- Combined effect of batching (6×) and caching (2.45×) yields ~19× reduction for fleet queries; residual 5-6s cold-start cost remains due to MCP stdio architecture.

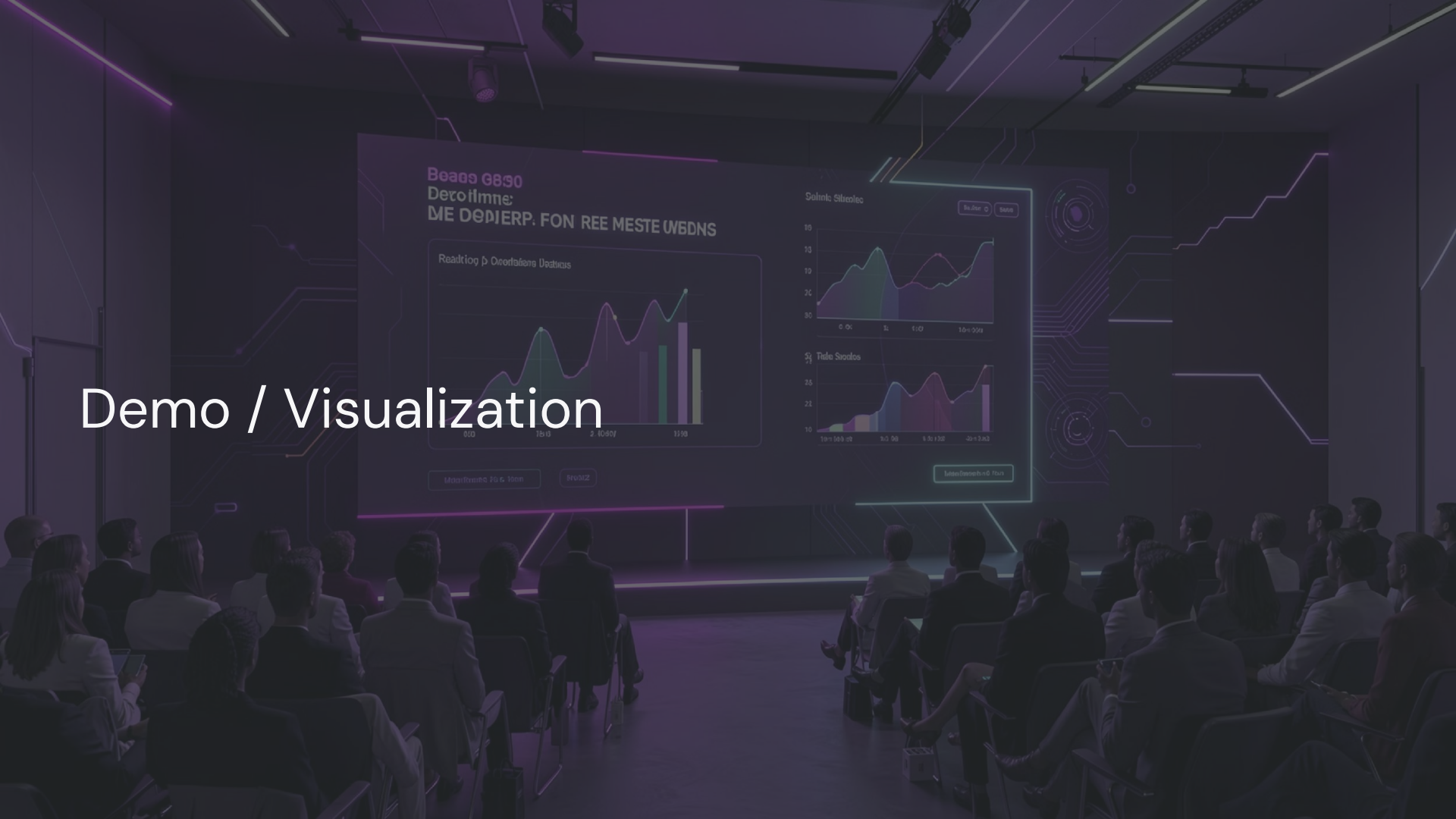
Profiler Evidence

Actual latency breakdown from Table 5 in the LaTeX report.

Stage	Total	Breakdown
Naive baseline	12.64s	5.66s fetch + 6.99s predict
Parallel fetch	11.18s	3.85s fetch + 7.33s predict
Graph precompile	10.88s	3.86s fetch + 7.03s predict
(Model Data) Batched predict	10.93s	3.83s fetch + 7.10s predict
Disk cache	3.83s	3.83s fetch + 0.002s predict

What to point at

- Bottleneck before optimization: TensorFlow model loading and inference dominates (7s per call)
- Optimization impact: Disk cache eliminates inference entirely by persisting .npz trajectories; parallel fetch reduces I/O by 32%
- Key insight: MCP stdio architecture spawns fresh subprocess per call, destroying in-memory state; only disk-backed persistence survives
- Residual cost: 3.8s CouchDB fetch cannot be eliminated without architectural change (persistent server)
- MCP batching vs. model batching: Consolidating 10 tool calls into 1 gave a 6× speedup by removing subprocess overhead; sending all 10 cells at once to the small CPU-bound model added only ~5%.



Demo / Visualization

Lessons Learned

Time budget: 1 minute. What surprised you? What didn't work? Be honest.

What worked

- Disk-backed caching proved essential: MCP's stdio architecture prevents in-memory state persistence, making disk the only viable cache layer across subprocess boundaries
- Tool-level batching (consolidating N cells into 1 call) reduced subprocess spawns and achieved 6× speedup; far more impactful than data-level batching
- Small language models (Llama 3.1 8B) can generate correct multi-step plans for complex fleet queries with well-designed few-shot examples and composable tool primitives

What didn't work

- Graph precompilation and batched TensorFlow inference: negligible gains (~5%) on small models.
- Sending all data at once to the model didn't help much on CPU because the bottleneck is subprocess spawn and I/O overhead, not compute. The real win from batching came from reducing MCP tool calls, not from batching data within a single call.

Limitations

Domain Shift

Hsu et al. trained on Severson LFP chemistry; NASA PCOE cells are NCA 18650. Absolute RUL predictions carry appreciable error. Statistical tools (which don't depend on pretrained weights) are more reliable. Fine-tuning on NCA data would address this but is beyond scope.

Context Window Constraints

4 of 15 scenarios exceeded Cerebras 32K token limit during synthesis. Fleet-wide time-series queries produce large array-valued outputs. Longer context window or summarization step would resolve.

Cold-Start Floor

~5–6s per MCP call unavoidable in stdio architecture (subprocess spawn, TensorFlow import, weight loading). Persistent server architecture would eliminate.

RUL Accuracy Evaluation

Ground truth curation for 10 deployed cells left as future work. Tool infrastructure demonstrated, but quantitative MAE evaluation requires complete charge/discharge profiles and documented end-of-life cycles.

Next Steps

Time budget: 30 seconds. Be specific — vague 'more experiments' is not next steps.

- Implement persistent MCP server (long-lived daemon process) instead of stdio subprocess model; measure in-memory cache benefits and compare to disk-backed approach; expected to eliminate 5-6s cold-start overhead
- Fine-tune the Hsu et al. model on NCA chemistry data (NASA PCOE cells) to address domain shift from LFP training data; quantify improvement in absolute RUL prediction accuracy
- Extend MCP architecture to multi-asset domains (HVAC, solar, wind) and measure generalization overhead; quantify cost of tool family composition across domains to validate extensibility claim

Teamwork & Contributions

Time budget: 30 seconds. Who did what. Required slide — graded individually.

Member	Primary contributions	Tools / artifacts
Siddharth Gowda	MCP server architecture, tool composition, batching optimization, CouchDB integration	src/servers/battery/main.py, src/couchdb/init_battery.py, src/agent/plan_execute/planner.py
Rushin Bhatt	CouchDB telemetry layer, data pipeline, parallel fetch optimization, profiling harness	src/servers/battery/couchdb_client.py, src/couchdb/couchdb_setup.sh, src/servers/battery/profiling/profile_scenario.py
Aryaman Agrawal	DNN model integration, tf_keras compatibility layer, disk cache implementation	src/servers/battery/preprocessing.py, src/servers/battery/model_wrapper.py, src/servers/battery/chemistries.yaml
Winston Li	Scenario design (15 scenarios), benchmark evaluation, results analysis, documentation	scenarios/local/battery_utterances.json, README.md, src/servers/battery/profiling/benchmark_optimizations.py

Collaboration: weekly meetings via Zoom, shared Slack channel, design meetings and reviews and critques in meetings and with mentor.

Thank you

Questions?

GitHub: https://github.com/siddharthgowda/AssetOpsBench

Orginal AssetOpsBench Repo: https://github.com/IBM/AssetOpsBench

Backup — Experimental Setup

Backup slide. Use for Q&A. Add more backup slides as needed.

Setting	Baseline	Optimized
Batch size	[1 cell per call]	[N cells per call]
Caching strategy	[None]	[Disk-backed .npz]
Model loading	[Fresh load per call]	[Cached in memory]
Parallel fetch	[Sequential]	[Thread pool, 4 workers]
Cells evaluated	[10 cells (B0005-B0056)]	[10 cells (B0005-B0056)]
Runs per config	[3 runs]	[3 runs]
Hardware	[Apple M4 Max, 14 cores, 36 GiB]	[Apple M4 Max, 14 cores, 36 GiB]
Software stack	[Python 3.12.10, TensorFlow 2.21.0, FastMCP, CouchDB]	[+disk cache, +parallel fetch, +batching]